

Taller de aula: leer una receta que todavía no corriste

Programación 3 - Tecnicatura Universitaria en Programación - UTN
FRM

Parte 1

1.1 Completa el inventario

Una fila por instrucción. En "Que la invalida" escribí el cambio más chico que obliga a volver a ejecutarla. En la última columna, qué pasaría si borraras esa línea del archivo.

Línea	¿Capa o metadato?	Que hace	Que la invalida (el cambio más chico)	Si la borro del archivo...
1	Hereda capas	Establece la imagen base sobre la cual se construye todo el entorno.	Cualquier cambio en los caracteres (como cambiar la versión).	Fallaría todo lo que dependa de python
2	Metadato	Fija el directorio de trabajo para todas las instrucciones siguientes.	Cualquier modificación en el texto de la ruta.	Las instrucciones siguientes se ejecutarán en el directorio raíz predeterminado de la imagen base
3 y 4	Capa	Se copia el archivo de requerimientos, y se instalan en el directorio.	Modificar cualquier carácter dentro de requirements.txt.	La ausencia de uno invalida al otro. Si no se copia el archivo, no se encontrará a la hora de querer instalar los requerimientos. Si no se instalan, el proyecto fallará al no tener las dependencias necesarias.
5 y 6	Capa	Agrega el archivo main del backend y de conexiones a la base de datos al sistema de archivos del directorio.	Modificar cualquier carácter dentro de estos archivos.	En una instrucción siguiente, se ejecutan estos archivos, por lo que su ausencia haría fallar a la instrucción del cmd, y la aplicación no se ejecutará correctamente.
7	Metadato	Documenta el puerto 8000, el del backend.	Modificar algún carácter de la instrucción, como el número de puerto (no haría fallar el proyecto, pero no se estaría documentando el puerto que usa la aplicación)	Principalmente fallarían herramientas de despliegue automático, que no sabrán a qué puerto enrutar el tráfico.

8	Metadato	Define el proceso de arranque, en este caso de ejecución.	Modificar algún carácter de la instrucción, especialmente sacar los corchetes "[]".	Al no tener la instrucción de arranque, se ejecuta el comando base de python-slim, que ejecuta el intérprete interactivo de python.
---	----------	---	---	---

1.2 Predecir la salida de docker history

Sin correr nada: de las ocho líneas, cuantas van a aparecer con un tamaño distinto de 0B? Nombralas y ordenalas de mayor a menor peso esperado, con una línea de justificación para la primera.

- Línea 4 (pip install requirements.txt): aunque se utiliza --no-cache-dir, se siguen guardando los archivos dentro del caché del Docker, solo que no se genera una copia en el caché de pip. El peso depende de las dependencias, pero es el peso agregado de cada archivo-dependencia.
- Línea 5 (Copy main.py)
- Línea 6 (Copy db.py)
- Línea 3 (Copy requirements.txt)

1.3 La instrucción que no esta

El archivo real del proyecto copia main.py y db.py por separado, en dos líneas, pudiendo escribir COPY . . en una sola. Da las dos razones por las que se eligió la versión larga. Una es de rendimiento y la otra es de otra categoría: decime cuál es cuál.

Rendimiento: al tener dos copy en la misma línea, si se hace una modificación en uno de los archivos invalida el caché de esa capa, obligando a copiar los dos archivos nuevamente. Esto provocaría que un pequeño cambio en un archivo requiera de reconstruir dos archivos, lo que lo volvería el doble de lento.

La otra podría ser para seguir el principio de **mínimo privilegio**. Darle a cada capa un archivo distinto, disminuyendo problemas posibles de seguridad que no se contemplaron en el momento.

Parte 2

2.1 Traza las tres construcciones

Complete la tabla marcando CACHED o EJECUTA en cada celda, y marca con un círculo, en las columnas B y C, la instrucción exacta donde se rompió la cadena

#	instrucción	A: cambio main.py	B: cambio requirements.txt	C: renombró workdir
1	FROM python:3.12-slim	CACHED	CACHED	CACHED
2	WORKDIR	CACHED	CACHED	EJECUTA 
3	COPY requirements.txt	CACHED	EJECUTA 	EJECUTA
4	RUN pip install ...	CACHED	EJECUTA	EJECUTA
5	COPY main.py .	EJECUTA	EJECUTA	EJECUTA
6	COPY db.py .	EJECUTA	EJECUTA	EJECUTA

2.2 Explicalo en una frase de ingeniería

Completa: "Renombrar el directorio de trabajo volvió a descargar todas las dependencias porque la clave del caché de una instrucción es _____ mas _____ , y porque cuando una capa se invalida _____ ."

Respuesta: Renombrar el directorio de trabajo volvió a descargar todas las dependencias porque la clave del caché de una instrucción es el texto literal de la instrucción, y porque cuando una capa se invalida todas las capas posteriores del Dockerfile se invalidan en cascada y deben reejecutarse obligatoriamente

2.3 El comando que muestra la verdad de adentro

Nombra el comando que permite leer, capa por capa, que instrucción creo cada una y cuanto pesa. Y explica que cosa muestra ese comando que ninguna lectura del Dockerfile puede mostrarte.

Respuesta: el comando que me permite leer capa por capa, que instrucción las creo a cada una y cuando pesa es "Docker History", este comando muestra cosas que ninguna lectura del dockerfile puede mostrar, esas cosas son:

- 1) el tamaño real de las capas en el disco y el peso de ellas en el archivo final

- 2) demuestra que las instrucciones de metadatos pesan exactamente 0B mientras que las instrucciones que modifican el sistema de archivos pesan en el disco
- 3) demuestra la persistencia de las capas ocultas y los archivos borrados, permite ver si se incluye un archivo en una capa anterior lo que significa que el archivo sigue usando espacio y existiendo dentro de la imagen

Parte 3

3.1 Ordena los doce pasos

Escribi las doce letras en el orden correcto de ejecucion.

k, c, l, h, j, f, e, b, a, i, d, g.

3.2 Marca los tres puntos de no retorno

Tres de esos pasos, puestos fuera de lugar, hacen dano de tres clases distintas: uno rompe la construccion en el acto, uno no rompe nada y te cuesta minutos en cada despliegue, y uno no se puede deshacer reescribiendo el archivo. Identificalos y decime, en cada caso, que error concreto los convierte en trampa.

1. **Trampa de privilegios de administrador (rompe la construccion | USER aplicacion):** si quitamos privilegios, o damos privilegios de usuario, los demás pasos que requieran de privilegios de administrador no se podrán ejecutar, lo que pararía la construcción.
2. **Trampa de invalidacion de cache (no rompe, cuesta minutos por cada despliegue | COPY [main.py](#) y RUN pip install requirements.txt):** si copiamos el archivo [main.py](#) antes de instalar los requerimientos, cada cambio que hagamos en el main requerirá desplegar nuevamente e instalar los requerimientos cada vez. Esto porque se haría cambios recurrentes al main, pero no tan frecuentemente a los requerimientos.
3. **Trampa de inmutabilidad de capas (cambio irreversible | .dockerignore):** en este archivo agregamos todo lo que no queremos que se suba a la imagen del docker. Si no lo configuramos bien, se podrían estar subiendo archivos importantes, como credenciales o api keys. Aunque luego se elimine y se modifique el .dockerignore, estos archivos seguirán siendo visibles en el historial de imágenes.

3.3 Los que no pesan y aun asi importan

Tres de los once pasos que si son instrucciones no crean ninguna capa. Identificalos y explica por que su posicion en el archivo igual importa, si no pesan nada.

1. **USER aplicacion:** aunque no pesa, es importante que se utilice después de instrucciones que requieran permisos de administrador, pero antes de desplegar la app, por temas de seguridad.
2. **WORKDIR /app:** no pesa, pero define el directorio donde se van a alojar los archivos de la aplicacion.. Es importante que vaya antes de instrucciones de tipo COPY o RUN que usen rutas relativas, porque si no se agregaran al directorio raíz, lo que puede generar problemas.

3. **La instrucción CMD:** es la instrucción que despliega la app, por lo que debe ir luego de USER aplicación, si no se desplegará con permisos root, lo que generaría problemas de seguridad.

Parte 4

4.1 Los cinco problemas

1. Línea 1: FROM python:latest
 - Corrección: Cambiar a una etiqueta de versión específica y liviana, por ejemplo: FROM python:3.12-slim.
 - Justificación: Usar latest rompe la reproducibilidad, ya que el contenido de la imagen base cambiará sin previo aviso al actualizarse el registro, además de incluir dependencias innecesarias que aumentan el tamaño y la superficie de ataque.
2. Líneas 3, 4 y 5: Mal orden de capas y comandos RUN apt-get separados
 - Corrección: Unificar la instalación de paquetes del sistema en una sola capa RUN al principio (usando && y --no-install-recommends), seguida de la limpieza del caché (rm -rf /var/lib/apt/lists/*), y colocar la copia e instalación de requirements.txt *antes* de copiar todo el código fuente.
 - Justificación: Si se ejecuta COPY . . antes de pip install, cualquier cambio en una línea de código invalidará la caché de la capa del COPY y obligará a reinstalar todas las dependencias en cada construcción.
3. Línea 6: Herramientas innecesarias en la imagen final (vim, git, curl)
 - Corrección: Quitar vim y git de la lista de paquetes a instalar (o usar una construcción multi-etapa si se requieren para compilar).
 - Justificación: Introducir editores de texto y control de versiones en la imagen de producción incrementa inútilmente el peso de la imagen y genera vulnerabilidades de seguridad.
4. Línea 7: ENV DATABASE_URL=postgres://postgres:tup2026@db:5432/tup
 - Corrección: Eliminar esa línea del Dockerfile e inyectar la variable de entorno en tiempo de ejecución (mediante .env, Docker Compose o la configuración de Easypanel).
 - Justificación: Acoplar datos sensibles y credenciales dentro del Dockerfile las expone de forma irreversible en el historial de capas de la imagen.
5. Línea 9: Sintaxis de CMD y dirección IP de escucha
 - Corrección: Cambiar a la sintaxis Exec (array) y escuchar en todas las interfaces: CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8000"].
 - Justificación: Al usar 127.0.0.1, Uvicorn solo aceptará peticiones provenientes del bucle local interno del propio contenedor, bloqueando el tráfico que provenga del exterior (o de otros contenedores/host). Además, usar la sintaxis de shell no pasa la señal SIGTERM correctamente al proceso.

4.2 La línea que no es un problema

- ¿Tiene razón en el hecho? Sí, en el aspecto técnico estricto tiene razón: la instrucción EXPOSE es meramente un metadato/documentación y no abre ni publica realmente el puerto en la interfaz de red del host por sí sola.

- ¿Por qué la decisión de borrarla es incorrecta? Es incorrecta porque sirve como documentación explícita del contrato del contenedor y es utilizada automáticamente por orquestadores, proxys inversos y herramientas de despliegue automatizado.
- ¿Quién consume esa línea? La consumen herramientas de automatización y orquestación (como Easypanel, Docker Compose, Traefik o AWS ECS) para descubrir automáticamente en qué puerto escucha la aplicación, así como los desarrolladores para entender rápidamente el punto de entrada sin leer el código.

Parte 5

5.1 Reconstrui la cadena de hechos

En seis líneas como máximo, contame que paso, en orden, desde que el grupo escribió el Dockerfile hasta que la contraseña terminó publicada. Sostene cada afirmación en una línea concreta de la evidencia.

Respuesta:

- 1) El grupo creó el dockerfile clonando la imagen con python:latest e instaló las dependencias con pip install -r requirements.txt
- 2) Copiaron los archivos del proyecto con COPY . . # buildkit, incluyendo por error el archivo .env que contenía la credencial DATABASE_URL
- 3) se dieron cuenta del error y con la línea RUN /bin/sh -c rm -f .env querian solucionarlo antes de que compile la imagen
- 4) usando una forma de intérprete generaron el comando de inicio: CMD ["sh" "-c" "uvicorn main:app --host 127.0..."]
- 5) Construyeron e iniciaron los contenedores de la API y la Web, pero como el backend estaba configurado en 127.0.0.1, el curl y el frontend recibieron respuestas vacías y errores.
- 6) Subieron la imagen a un repositorio público; un atacante descargó la imagen, extrajo el archivo TAR con docker save y recuperó la contraseña inspeccionando las capas históricas no eliminadas.

5.2 Las tres contradicciones aparentes

La evidencia parece contradecirse tres veces. Explica cada caso:

- (i) el log dice Uvicorn running y no hay un solo error, pero curl no obtiene respuesta;
- (ii) docker exec ls -a no muestra ningun .env dentro del contenedor, y sin embargo la credencial se extrae de la imagen;
- (iii) el contenedor del frontend arrancó sin errores y la calculadora igual no funciona.

Respuesta:

- 1) esta contradicción se da porque uvicorn esta escuchando en el puerto 127.0.0.1 y al intentar acceder desde afuera con: "curl <http://localhost:8000/api/salud>" el tráfico no puede ingresar porque la aplicación no escucha la interfaz pública en el puerto 0.0.0.0.
- 2) Esta contradicción se da porque la línea COPY . . # buildkit copio de forma persistente todos los archivos, incluyendo el env., el grupo al darse cuenta del error usa el comando: RUN /bin/sh -c rm -f .env para "eliminar" ese archivo, el problema es que este comando en lugar de borrar por completo el env. creó una capa máscara por encima que le oculta este archivo al sistema operativo superior, y a la hora de ejecutar docker exec se muestra la capa final de la superposición, escondiendo el archivo env. aún existente, y cuando se ejecuta docker save, se muestran todos los archivos de todas las capas permitiendo al atacante encontrar la información que necesite.
- 3) Esta contradicción ocurre porque el contenedor del frontend (NGINX) entrega correctamente los archivos estáticos del HTML y del JavaScript, el problema ocurre cuando el código del javascript trata de comunicarse con la API del backend que no

responde debido a la primera contradicción que indica que está mal enrutada en al red (usa el puerto 127.0.0.1 en lugar del 0.0.0.0.)

5.3 Los principios violados

Nombra los principios del capítulo que el grupo violó y en qué decisión concreta. Uno de ellos ya lo viste en la Clase 2 y aparece acá aplicado dentro de la imagen: decime cual y como se traduce.

Respuesta:

Rotación inmediata de credenciales (Seguridad): Un secreto que tocó una capa pública debe considerarse "quemado" e invalidarse en la base de datos de inmediato.

Separación de configuración y código (Metodología 12 Factors): La URL de la base de datos y credenciales no deben empaquetarse en la imagen; deben inyectarse mediante variables de entorno en tiempo de ejecución.

Uso de .dockerignore: Excluir archivos sensibles (.env) del contexto de construcción para evitar filtraciones.

Principio de Mínimo Privilegio y Superficie de Ataque (visto en Clase 2): Aplicado a no empaquetar secretos expuestos ni usar etiquetas ambiguas (latest).

5.4 Las cuatro correcciones, en orden

Escribí las cuatro medidas que aplicarías, en el orden en que las aplicarías. Justifica el orden, no solo el contenido: una de las cuatro no puede esperar a las otras tres.

respuesta:

1.Rotar las credenciales inmediatamente:

Cambiar la contraseña de la base de datos de inmediato en el servidor de producción.

Justificación: Esta medida no puede esperar a las demás porque la contraseña actual ya fue expuesta en una imagen pública y cualquier otra corrección es inútil si el acceso a la base de datos sigue estando expuesto a ataques. para verificar la corrección del problema alguno de los integrantes del grupo debería intentar conectarse a la base de datos usando la contraseña anterior y verificar que se le niegue el acceso

2.Agregar el .env al archivo .dockerignore:

Lo segundo que deberíamos hacer es añadir .env dentro del archivo .dockerignore del backend y eliminar la línea RUN rm -f .env del Dockerfile. Este paso debe hacerse a continuación del primero y antes de compilar nuevamente la imagen, para que cuando se ejecute el comando COPY . . # buildkit no se copie el archivo .env que guarda la nueva contraseña y evitar el problema de la exposición de la base de datos, para verificarlo uno de los integrantes debería ejecutar docker build y revisar con docker history que ninguna capa intermedia arrastre el archivo .env.

3.Corregir la interfaz de escucha a 0.0.0.0:

Este paso se realiza en esta posición porque primero había que garantizar la seguridad del empaquetado, modificando el CMD del Dockerfile para que escuche en todas las interfaces: CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8000"]. para que el servicio sea accesible desde el exterior y el frontend funcione correctamente, para verificar que funcione

bien habría que reconstruir, iniciar el contenedor y ejecutar curl `http://localhost:8000/api/salud`, obteniendo una respuesta correcta.

4. Usar la forma de ejecución (Exec Form) en CMD y fijar la versión base:

esta es la última corrección a realizar ya que tanto las medidas de seguridad como la conectividad de la api ya fueron reparadas con éxito, en este caso habría que escribir el CMD en formato array de JSON con comillas dobles y reemplazar `python:latest` por una versión específica como `python:3.12-slim`. Esto evita cierres abruptos del servidor y que las futuras construcciones sean reproducibles al no depender de latest. Para verificar que la corrección funcione habría que detener el contenedor con `time docker stop tup-api` y comprobar que el tiempo de apagado sea inferior a 2 segundos en lugar de 10 segundos.

Parte 6

Servicio	Imagen base con etiqueta exacta	Qué se copia y en qué orden	Qué NO entra (.dockerignore)	Qué decide el entorno	Puerto interno
Frontend	node:20-alpine (o nginx:alpine si es estático)	1. package*.json 2. RUN npm install 3. Código fuente (src/, public/)	.git, node_modules/, .env, .env.local, dist/, README.md	VITE_API_URL (URL pública de la API para las peticiones cliente)	80 (o 5173 / 3000)

API	python:3.12-slim	1. requirement s.txt 2. RUN pip install ... 3. main.py, db.py y demás archivos del código	.git, .venv/, __pycache_ _/, .env, *.pyc, tests/	DATABASE_URL, SECRET_KEY, PORT, DEBUG	8000
Base de datos	postgres:16-alpine	Ninguno (se usan volúmenes para la persistencia de los datos en tiempo de ejecución)	N/A (no suele requerir Dockerfile propio)	POSTGRES_USER , POSTGRES_PASSWORD, POSTGRES_DB	5432

6.1 La decisión que arrastra a todas las demás (5 puntos)

- Celda: La imagen base con etiqueta exacta (ej. python:3.12-slim).
- ¿Por qué? Porque define el sistema operativo subyacente, el gestor de paquetes (apt, apk), el intérprete preinstalado y la estructura del sistema de archivos. Si se cambia la imagen base (por ejemplo, de python:3.12-slim a python:3.12-alpine), cambian los comandos del sistema (apt vs apk), la forma en que se compilan ciertas librerías y la compatibilidad con las dependencias.
- ¿Qué problema te evita fijarla hoy? Evita el problema del "drift" de entorno o fallos inesperados de construcción en producción: garantiza que tanto en las máquinas de desarrollo como en los servidores de despliegue (Easypanel) se utilice exactamente el mismo entorno ejecutable, previniendo que una actualización automática cambie la versión subyacente y rompa el proyecto.

6.2 La línea que te vas a agradecer (5 puntos)

- Respuesta técnica:
"Si incluimos el archivo .env dentro de la imagen Docker, las credenciales, contraseñas y claves secretas quedarán grabadas permanentemente en las capas de la imagen. Cualquier persona o herramienta con acceso a la imagen (o al registro de imágenes) podrá extraer el archivo e inspeccionar los secretos mediante comandos como docker history o analizando las capas subyacentes, comprometiendo la seguridad de todo el sistema."
- Alternativa concreta:
"Utilizaremos un archivo .env.example en el repositorio para documentar qué variables necesita el sistema sin incluir valores reales. En el servidor (Easypanel) inyectaremos las variables de entorno reales directamente en la sección de configuración de entorno del contenedor al momento del despliegue, manteniéndolas completamente desacopladas de la imagen."